

Array

Arrays power most real-world JavaScript data work. Mastering the core methods and loops is essential for exams, interviews, and everyday productivity. Use this Cheat-Sheet to refresh quickly.



JS

Loops

for (indexed) → Full control (**break** / **continue**)

```
for (let i = 0; i < arr.length; i++) { const v = arr[i]; }
```

for...of → Iterate values

```
for (const v of arr) { /* ... */ }
```

entries() → Index + value via for...of

```
for (const [i, v] of arr.entries()) { /* ... */ }
```

forEach(fn) → Side effects only; no early break

```
arr.forEach(v => { /* ... */ });
```

while → Loop while condition is true

```
let i = 0; while (i < arr.length) { i++; }
```

do...while → Runs at least once

```
let i = 0; do { i++; } while (i < arr.length);
```

for...in → Enumerable keys; avoid for arrays

```
for (const k in arr) { /* */ }
```

for await...of → Async iteration

```
for await (const v of stream) { /* ... */ }
```

Search & Test

find(predicate) → First matching element or **undefined**

```
[1,4,6].find(x => x > 3) // 4
```

findIndex(predicate) → Index of first match or **-1**

```
[1,4,6].findIndex(x => x > 3) // 1
```

findLast(predicate) → Last matching element or **undefined**

```
[1,4,6].findLast(x => x > 3) // 6
```

findLastIndex(predicate) → Index of last match or **-1**

```
[1,4,6].findLastIndex(x => x > 3) // 2
```

some(predicate) → Any element passes?

```
[1,2,3].some(x => x > 2) // true
```

every(predicate) → All elements pass?

```
[2,4,6].every(x => x % 2 === 0) // true
```

indexOf(value) / lastIndexOf(value) → Position or **-1**

```
['a','b','a'].lastIndexOf('a') // 2
```

includes(value) → Presence check

```
[1,2].includes(2) // true
```

Note the one array method or loop you find most useful in your daily work, drop it below to make it stick.

Brought to you by



Learn more at certificates.dev/javascript

Creators of the



JavaScript Certification

Array

Arrays power most real-world JavaScript data work. Mastering the core methods and loops is essential for exams, interviews, and everyday productivity. Use this Cheat-Sheet to refresh quickly.

JSJS

Loops

for (indexed) → Full control (`break` / `continue`)

```
for (let i = 0; i < arr.length; i++) { const v = arr[i]; }
```

for...of → Iterate values

```
for (const v of arr) { /* ... */ }
```

entries() → Index + value via for...of

```
for (const [i, v] of arr.entries()) { /* ... */ }
```

forEach(fn) → Side effects only; no early break

```
arr.forEach(v => { /* ... */ });
```

while → Loop while condition is true

```
let i = 0; while (i < arr.length) { i++; }
```

do...while → Runs at least once

```
let i = 0; do { i++; } while (i < arr.length);
```

for...in → Enumerable keys; avoid for arrays

```
for (const k in arr) { /* */ }
```

for await...of → Async iteration

```
for await (const v of stream) { /* ... */ }
```

Search & Test

find(predicate) → First matching element or `undefined`

```
[1,4,6].find(x => x > 3) // 4
```

findIndex(predicate) → Index of first match or `-1`

```
[1,4,6].findIndex(x => x > 3) // 1
```

findLast(predicate) → Last matching element or `undefined`

```
[1,4,6].findLast(x => x > 3) // 6
```

findLastIndex(predicate) → Index of last match or `-1`

```
[1,4,6].findLastIndex(x => x > 3) // 2
```

some(predicate) → Any element passes?

```
[1,2,3].some(x => x > 2) // true
```

every(predicate) → All elements pass?

```
[2,4,6].every(x => x % 2 === 0) // true
```

indexOf(value) / lastIndexOf(value) → Position or `-1`

```
['a','b','a'].lastIndexOf('a') // 2
```

includes(value) → Presence check

```
[1,2].includes(2) // true
```

Note the one array method or loop you find most useful in your daily work, drop it below to make it stick.

Brought to you by



Learn more at certificates.dev/javascript

Creators of the

